# Heurion 2.0: A Contract-Guarded Dual-Surface Architecture with Self-Evolving Memory for Zero-Hallucination Clinical Research

**Heurion Research Team**
Correspondence: `research@heurion.ai`

October 2026

**Abstract**

Autonomous large language model (LLM) agents have achieved strong results in software engineering through free-form shell execution and direct file editing. However, applying this "script-and-audit" workflow to clinical and scientific research causes serious failures: broken formatting in Office Open XML (`.docx` and `.pptx`) manuscripts, silent overwriting of concurrent human edits, fabricated literature identifiers (DOIs and PMIDs) or unverified statistical claims, accidental exposure of Protected Health Information (PHI), and repeated guard failures across sessions.

We present **Heurion 2.0**, a collaborative clinical research and scientific authoring platform built around two core ideas: a *Contract-Guarded Dual-Surface Document Engine* and a *Three-Layer Self-Evolving Memory System*. First, instead of letting agents edit raw files on disk with ad-hoc scripts, Heurion 2.0 keeps a single Conflict-free Replicated Data Type (CRDT) document state (ProseMirror + Yjs) with stable block IDs (`{#id}`). From this shared state, it projects a rich-text or slide editor for human researchers and a compact Annotated Markdown view with guarded editing tools for AI agents. This design enables per-turn selective undo of AI edits without disturbing human changes in the same paragraph, as well as 100% byte-identical OOXML export for untouched blocks. Second, to prevent hallucinated references, pre-write guards block raw inline DOIs and PMIDs while a multi-agent verification pipeline checks manuscript claims against PubMed abstracts and PubMed Central (PMC) full texts. Third, a clinical data pipeline enforces deterministic PHI redaction, pseudonymous subject IDs (`P001`), read-only patient health records, and strict permission parity between the web UI and AI tools. Finally, a three-layer self-evolving memory system combines side-effect-free pre-flight validation, a dual-track learning engine that acquires domain conventions through exemplar corruption-and-reconstruction (Track A) and operational rules from verifier feedback without hand-written templates (Track B), and background profile distillation. In empirical benchmarks and live `deepseek-flash` multi-session evaluations, Heurion 2.0 achieves 100% byte identity on unmodified OOXML blocks, sub-millisecond guard validation (0.168 ms) and edit commits (0.351 ms), a 62.5% overall and 75.0% held-out reduction in guard rejections with zero hand-crafted rule templates, and cross-domain skill transfer across clinical manuscripts, biostatistical plans, and presentation decks with zero negative transfer.

**Keywords:** Collaborative AI Agents, CRDT Co-Editing, Clinical Research Informatics, Citation Verification, Self-Evolving Memory, Skill Training

## 1 Introduction

Clinical and biomedical research workflows require strict accuracy, traceability, and privacy compliance. A typical clinical study spans longitudinal patient record curation, cohort screening, biostatistical analysis (such as Baseline Table 1 summaries, Kaplan–Meier survival curves, Cox regression, and CONSORT flowcharts), and collaborative authoring of peer-reviewed manuscripts

(`.docx`) and presentation decks (`.pptx`) [1, 2]. While recent agentic systems excel at code generation [3, 4] and open-domain literature review [5, 6], deploying autonomous agents into day-to-day clinical research exposes four practical bottlenecks:

1. **The Free-Form File Scripting Trap:** First-generation document agents treat `.docx` and `.pptx` files as opaque disk archives, generating Python scripts (e.g., `python-docx`) to rewrite them and running post-hoc quality checks afterward. In practice, script-based rewrites strip publisher XML attributes, lose paragraph IDs (`w14:paraId`) during LibreOffice layout passes, introduce subtle punctuation drift during full-paragraph replacements, and make it impossible to undo an AI turn once a human co-author edits the same section.
2. **Fabricated Citations and Unverified Clinical Claims:** LLMs frequently invent plausible-looking DOIs, PMIDs, hazard ratios ($HR$), and $p$-values [7, 8]. Prompting alone cannot guarantee that every inline citation resolves to a real publication or that reported numbers match the cited paper.
3. **Patient Privacy (PHI) Risks and Permission Gaps:** Clinical investigators work with raw medical records containing Protected Health Information (patient names, national ID numbers, admission codes, and phone numbers). Sending unredacted text to external LLMs violates healthcare privacy regulations [9]. At the same time, giving agents direct database or shell access bypasses application-level access controls.
4. **Cross-Session Amnesia and Memory Interference:** When an agent triggers a guard error (such as editing a paragraph that carries an open review comment or merging non-adjacent blocks) or when a clinician reverts an AI turn, stateless agents repeat the same mistake in future sessions. Dumping all past observations into a single flat memory list mixes subjective writing style preferences with hard tool constraints, filling the prompt budget with irrelevant rules [10, 11].

### 1.1 Lessons from First-Generation Post-Hoc Auditing (Heurion v1)

Before building Heurion 2.0, we deployed a first-generation system (*Heurion v1 / S-Series*) where an agent ran Python scripts in a workspace shell to unpack, modify, and repackage `.docx` files while a post-turn supervisor checked text survival rates, paragraph drift, and DOI validity. Real-world use showed that *post-hoc auditing happens too late*: by the time an audit script catches a broken XML run or a hallucinated citation, the file on disk is already corrupted, forcing full-file rollbacks that wipe out concurrent browser edits.

### 1.2 The Heurion 2.0 Architecture and Contributions

Heurion 2.0 replaces post-hoc file scripting with *pre-write structural guards over a shared CRDT document core*, paired with a *cross-domain self-evolving memory system*. Our main contributions are:

- **Contract-Guarded Dual-Surface CRDT Engine (Section 3):** We build a unified ProseMirror + Yjs document core with persistent block IDs (`{#id}`), block-level optimistic concurrency checks (`base_rev`), minimal-diff XML updates that preserve inline formatting and comment anchors, per-turn selective CRDT undo, and 100% byte-preserving OOXML export for unmodified blocks.
- **Zero-Hallucination Evidence & Clinical Privacy Pipeline (Section 4):** We block raw inline DOIs and PMIDs before any write occurs, requiring registration through Crossref, PubMed, and OpenAlex into atomic citation nodes (`[@c:id]`). A multi-agent verifier

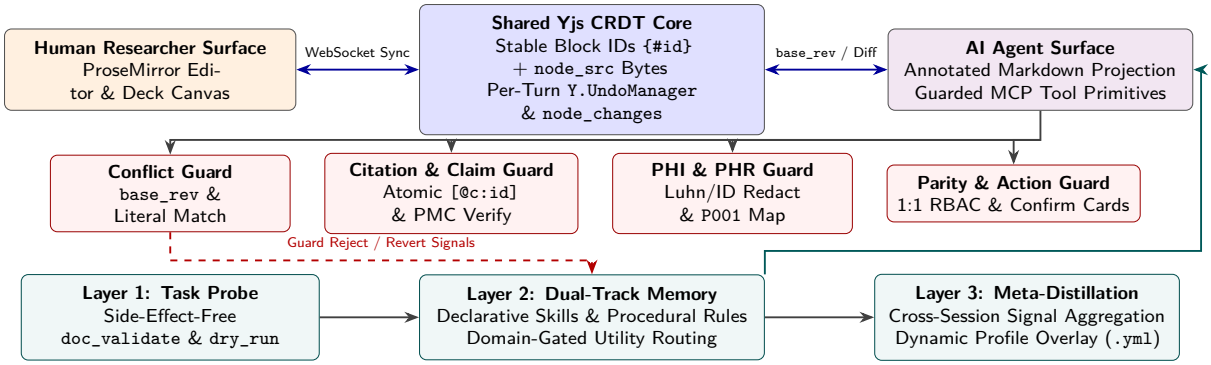



Figure 1: Architecture of **Heurion 2.0**. Top: Human researchers and AI agents work on a shared Yjs CRDT document and slide core through synchronized views. Middle: All AI edits pass through four deterministic pre-write guards before touching the CRDT state. Bottom: Guard rejections, tool recovery trajectories, and gold exemplars drive a three-layer self-evolving memory system that routes task-relevant rules into future turns.

checks sentence-level claims against PubMed abstracts and PMC full texts, while a multi-tenant clinical sandbox enforces checksum-verified PHI redaction, pseudonymous subject IDs (`P001`), and reproducible biostatistical execution.

- **Cross-Domain Three-Layer Self-Evolving Memory (Section 5):** We design a three-timescale adaptation loop: (Layer 1) side-effect-free pre-flight validation probes, (Layer 2) a domain-agnostic dual-track memory engine that combines self-supervised corruption-and-reconstruction skill training (Track A) with zero-template procedural rule synthesis and backward credit assignment (Track B), and (Layer 3) background distillation into dynamic agent profile overlays.
- **Verified Human-AI Permission Symmetry (Section 6):** We enforce and automatically test in CI that every REST API endpoint either maps 1-to-1 to an in-process Model Context Protocol (MCP) tool running under the user's exact identity and role permissions, or is explicitly listed in an audited human-only exclusion registry, with destructive actions gated by interactive confirmation cards.

## 2 System Overview and Design Principles

Figure 1 shows the overall architecture of Heurion 2.0. The platform separates *structured document and slide editing* from *sandboxed statistical computation*, connecting human researchers and AI agents through a shared Yjs CRDT state.

Three practical design principles guide the system:

**Principle 1: Pre-Write Structural Safety.** Every batch of AI edit operations is validated against the live CRDT tree *before* any state change occurs. A whole-block replacement is allowed only if no human user has edited that block since the revision (`base_rev`) the agent read. Likewise, any edit that introduces an unregistered citation identifier, breaks a table grid, or deletes an active review comment anchor without acknowledgment is rejected atomically. When a batch is rejected, the document remains untouched and the guard returns a structured error code, a diagnostic hint, and the latest block text so the agent can recover immediately.

**Principle 2: Human-AI Permission Symmetry.** An AI agent should neither hold superuser privileges over the database nor be restricted to a toy subset of actions. In Heurion 2.0, every REST API route exposed to the browser UI is either paired 1-to-1 with an MCP tool

that executes in-process under the logged-in user's exact session token and tenant permissions, or is explicitly listed in an audited human-only exclusion list (such as password changes or final confirmation endpoints). High-risk operations triggered by an agent pause and render a confirmation card in the user's chat stream before executing.

**Principle 3: Separating Domain Style Preferences from Operational Rules.** Mixing subjective writing preferences (e.g., "use passive voice in Methods") with objective tool-use rules (e.g., "move non-adjacent blocks together before merging") causes severe prompt interference. Heurion 2.0 keeps these two tracks separate:

- **Declarative Memory (Track A):** Stores domain conventions, writing styles, and analytical preferences. New declarative skills are proposed as candidates—either from user edits or from self-supervised exemplar training—and require human approval (or explicit activation) before taking effect.
- **Procedural Harness Memory (Track B):** Stores objective operational rules for passing pre-write guards and tool verifiers. Procedural rules evolve automatically from deterministic runtime failures and intra-turn recovery trajectories without requiring manual curation in the UI.

# 3 Dual-Surface Co-Editing and Byte-Preserving Document Engine

A key lesson from first-generation document agents is that Office Open XML (`.docx` and `.pptx`) files are delivery archives rather than collaborative editing data structures. When an agent rewrites a `.docx` file on disk while a human views the manuscript in a browser, the two sides immediately diverge.

## 3.1 Shared CRDT Core with Lifecycle-Stable Block Addressing

In Heurion 2.0, every manuscript and slide deck lives in a server-authoritative Yjs CRDT document (`Y.Doc`) backed by SQLite snapshots and an append-only revision log (`doc_commits`). Every block node (heading, paragraph, list item, table, figure, or slide box) carries a short, persistent ID (`{#id}`). When a human splits a paragraph or pastes content in the browser, the server automatically detects duplicate or missing IDs and assigns fresh unique IDs inside a system CRDT transaction, keeping block addressing consistent across the document's lifetime.

From this single CRDT core, Heurion 2.0 projects two synchronized views:

1. **Human Surface:** A collaborative ProseMirror rich-text editor for manuscripts and an interactive 16:9 slide canvas for presentation decks, synchronized in real time over WebSockets.
2. **Agent Surface:** A compact *Annotated Markdown* view generated on demand via `doc_read` and `slide_read`. Each block ends with its stable address `{#id}`, inline citations appear as atomic tokens `[@c:cite_id]`, figures as `![alt](asset:id)`, and open inline review comments as `{#c:comment_id}`.

## 3.2 Block-Level Conflict Guards and Minimal-Diff Updates

To prevent an AI turn from overwriting concurrent human edits, every `doc_edit` call includes the `base_rev` revision number read by the agent. The server checks the `node_changes` log for any blocks edited by a human since `base_rev`:

- **Whole-Block Edits (`replace_block`, `delete`):** If a human has modified the target block since `base_rev`, the edit is rejected with `conflict_user_edited` and returns the human's latest text. Blocks with unresolved track-changes suggestions are similarly protected by `pending_suggestion`.
- **Surgical Phrase Replacement (`replace_text`):** For targeted phrase edits inside a paragraph, the literal `find` string serves as a natural concurrency check: if the human edited a different sentence in the same paragraph, `find` still matches uniquely and succeeds; if the human changed the target phrase itself, `find` fails cleanly with `text_not_found`.
- **Mark-Preserving Minimal Diffs:** Importantly, `replace_text` does not re-parse the whole paragraph from plain text, which would strip inline bold/italic formatting, `[@c:id]` citation nodes, and comment highlights outside the edited span. Instead, it locates the character offsets in the ProseMirror node's text stream and replaces only that slice, updating only the changed XML children between the common prefix and suffix in the underlying Yjs fragment.

### 3.3 Per-Turn Selective Undo in CRDT Space

In collaborative writing, a researcher may ask the AI to revise three paragraphs, manually edit the second paragraph, and then decide to undo the AI's turn. A standard linear undo (`Ctrl+Z` or snapshot restore) would wipe out the researcher's manual edits.

Heurion 2.0 solves this with a two-tier *Per-Turn Selective Undo* (`revertTurn`):

1. **In-Memory CRDT Origin Tracking:** When an AI turn commits edits, the server tags the Yjs transaction with a turn-specific origin and attaches a scoped `Y.UndoManager`. Calling `revertTurn` undoes only the CRDT insertions and deletions made by that specific AI turn, leaving concurrent human edits—even inside the same paragraph—untouched.
2. **Persistent Block-Level Fallback across Restarts:** Before an AI turn's first write, the server saves the pre-turn Yjs state vector in SQLite. If the server restarts before `revertTurn` is called, the engine reconstructs the pre-turn tree, reverts all AI-edited blocks that have not been touched by a human since that turn, and safely skips any block subsequently edited by the user.

### 3.4 Byte-Preserving OOXML Patch Export

Clinical manuscripts are often imported from journal Word templates containing custom styles, headers, macros, and embedded objects. During `.docx` or `.pptx` import, Heurion 2.0 stores the original archive and caches the raw XML slice (`<w:p>` or `<w:tbl>`) of every imported block in `node_src`. On export, any block whose ProseMirror structure is unchanged from import is written back using its exact original XML bytes; only blocks actually modified by the user or AI are re-serialized, preserving the original paragraph properties (`<w:pPr>`).

## 4 Zero-Hallucination Evidence Loop and Privacy-Preserving Clinical Pipeline

### 4.1 Pre-Write Citation Guard and Atomic Citation Nodes

To stop fabricated references at the source, Heurion 2.0 blocks agents from writing raw bibliographic IDs or hand-numbered brackets in manuscript text. Before any edit is applied, the pre-write validator scans the new text for bare DOIs (`10.\d{4,9}/...`), bare PMIDs, or unlinked bracketed reference numbers. Any match triggers `citation_not_registered` and aborts the write.

Instead, the agent must call `insert_citation`, which queries Crossref, NCBI PubMed E-utilities, and OpenAlex, verifies that the publication exists, stores its canonical metadata in the document's `citations` table, and returns a registered token `[@c:cite_id]`. In ProseMirror, `[@c:cite_id]` is an atomic inline node that numbers itself automatically and formats the bibliography in Vancouver, AMA, APA, MLA, IEEE, or GB/T 7714 style.

### 4.2 Multi-Agent Sentence-Level Claim Verification

Registering a real paper does not prevent an LLM from misstating its findings (for example, citing a real trial but flipping the hazard ratio). Heurion 2.0 pairs specialist subagents (`literature_researcher`, `biostatistician`, and `fact_checker`) with an automated claim verification pipeline (`verify_claims`):

1. **Claim Extraction:** The verifier scans all document blocks, splits text into sentences, and selects (a) sentences containing `[@c:cite_id]` citations and (b) uncited sentences containing quantitative clinical assertions (percentages, $p$-values, confidence intervals, $HR/OR/RR$, or sample sizes).
2. **Evidence Retrieval:** For each cited paper, the verifier fetches the PubMed abstract and, when available via PubMed Central (PMC) or uploaded PDFs, the full-text sections.
3. **Four-Way Verdict & Inline Review Comments:** Each extracted claim receives one of four verdicts: `supported`, `contradicted`, `unsupported` (the cited paper does not mention the metric), or `uncited_stat`. Flagged claims are attached directly to the offending sentence as inline ProseMirror review comments with excerpts from the source paper and one-click actions (apply suggested rewrite, swap citation, or mark as a *Clinical Experience Exemption*).

### 4.3 Multi-Tenant PHI Redaction and Reproducible Clinical Sandbox

To connect clinical data analysis with manuscript writing without leaking patient identities:

- **Per-Tenant Database Isolation & Envelope Encryption:** Each organization runs on a physically separate SQLite database file with per-tenant Data Encryption Keys wrapped by a master Key Encryption Key.
- **Deterministic PHI Scanner:** Before any clinical note or OCR lab report enters LLM context, deterministic scanners redact 18-digit national ID numbers (validated via ISO 7064 MOD 11-2 checksum), card numbers (Luhn check), phone numbers, medical record numbers, and co-occurring patient name + sensitive diagnosis phrases.
- **Pseudonymous Subject IDs & Read-Only Health Records:** Inside research cohorts, real patient identifiers are replaced with pseudonymous codes (`P001`, `P002`, ...). In addition, a dedicated guard enforces strict read-only access on patient-uploaded Personal Health Records (PHR) so AI tools can never alter primary clinical evidence.
- **Sandboxed Biostatistical Execution:** Statistical computations are delegated to isolated tools and the `biostatistician` subagent, producing reproducible Baseline Table 1 summaries, Kaplan–Meier survival curves, CONSORT flowcharts, E-value sensitivity analyses, clinical prediction nomograms, and MONAI 3D medical image segmentations.

## 5 Three-Layer Self-Evolving Agent Architecture

A central practical question in guarded agent environments is how an agent learns from mistakes and domain exemplars without relying on human prompt engineering or hard-coded error dictionaries. As shown in the bottom tier of Figure 1, Heurion 2.0 organizes self-evolution across three timescales.

Table 1: Concrete examples of how Layer 2 learns domain conventions (Track A) and operational guard rules (Track B) without hand-crafted rule templates.

| **Track & Domain** | **Input Signal / Degraded Skeleton** | **Contrast vs. Gold Exemplar or Recovery** | **Synthesized Reusable Skill / Rule** |
|---|---|---|---|
| **Track A: Exemplar** (`biostats`) | *Degraded skeleton:* "Compare baseline continuous and categorical variables; run multivariable survival regression." | Initial draft uses mean ± SD and chi-square everywhere; gold SAP specifies Shapiro–Wilk check, Median [IQR], Fisher's exact test ($E < 5$), and Schoenfeld residuals. | `[domain:biostats]` Test normality first; report skewed variables as Median [IQR] with Mann–Whitney U, and switch to Fisher's exact test when expected cell count $< 5$. |
| **Track A: Exemplar** (`deck`) | *Degraded skeleton:* Flat single-paragraph dump of ICU length-of-stay numbers and subgroup comparisons. | Initial slide uses a generic noun title ("Results") and 6 crowded bullets; gold slide uses an action title and a two-column chart layout. | `[domain:deck]` Use a single-sentence Action Title stating the primary finding, and pair group comparisons in a two-column layout ($\leq 4$ bullets + chart). |
| **Track B: Zero-Template** (non-adjacent merge) | *Failed call:* `merge(p_d1, p_d3)` rejected because paragraph `p_d2` sits between them. | *Intra-turn recovery:* Agent calls `move(p_d3, after=p_d1)` followed by `merge(p_d1, p_d3)` in the same batch. | On `invalid_structure`: verify target blocks are adjacent before merging; if separated, execute `move` first and then `merge` in the same batch. |
| **Track B: Credit Assignment** (open comments) | *Implicit prompt:* "Make the opening sentence of Methods more concise in 'Cohort B' " (rule existed but was not routed; agent used `replace_block`). | *Diagnosis:* Rule was absent from active set (*Routing Miss*) and lacked an `ack_comments` fallback (*Rule Insufficiency*). | Strips cohort name 'Cohort B', adds `Methods \| opening sentence` to trigger pattern, and appends `ack_comments` + thread reply fallback to the rule body. |

### 5.1 Layer 1: Task-Level Pre-Flight Probe (`doc_validate` & `dry_run`)

Within a single turn, agents can call `doc_validate` (or pass `dry_run: true` to `doc_edit`) before committing complex multi-block edits. The probe runs all conflict guards, citation checks, comment-anchor checks, and table validators against an in-memory clone of the ProseMirror tree without advancing the revision counter or touching the undo stack. If a check fails, the probe returns the error code, a recovery hint, and the current block text, letting the agent fix its tool call before committing.

### 5.2 Layer 2: Cross-Domain Dual-Track Self-Evolution

Across turns and sessions, Layer 2 uses a domain-agnostic evolution kernel backed by pluggable domain adapters (`manuscript`, `biostats`, `deck`, and `general_mcp`). It combines two complementary learning tracks (illustrated with concrete examples in Table 1):

**Track A: Learning Domain Conventions via Corruption and Reconstruction.** Waiting for users to manually rewrite AI drafts leaves a new workspace with a cold-start gap. Drawing inspiration from self-supervised skill training via semantic diffusion [15], Track A lets the platform learn reusable domain skills directly from existing high-quality documents (such as

published clinical manuscripts, verified Statistical Analysis Plans, or executive slide decks) in three steps:

1. **Forward Corruption:** A domain adapter strips expert formatting, rhetorical transitions, methodological checks, or slide layout tags from a gold exemplar, leaving only a plain list of raw facts or analysis goals.
2. **Skill-Conditioned Reconstruction:** The agent attempts to reconstruct the full professional text or analysis plan from the degraded skeleton using only its currently active skills.
3. **Contrastive Skill Update & Safety Filtering:** By comparing its reconstruction against the original exemplar, the engine identifies what domain knowledge was missing or too vague and proposes structured skill updates (`create`, `update`, `narrow`, or `merge`). Before any candidate skill is saved, an *Instance–Skill Decoupling Guard* and PHI scanner automatically block rules that memorize study-specific numbers (such as sample size $N = 312$ or hazard ratio $HR = 0.68$), database primary keys, or patient identifiers.

**Track B: Zero-Template Procedural Synthesis and Backward Credit Assignment.** Earlier versions of procedural memory relied on a static lookup dictionary that mapped known guard error codes to hand-written rule strings. To achieve true domain generalization, Heurion 2.0 removes all hand-written rule templates. Whenever a pre-write guard or domain tool verifier rejects an action and the agent recovers later in the same turn, the runtime records the failed call, the verifier's diagnostic hint, the successful recovery call, the user prompt, and the list of rules that were active in that turn.

Once an error pattern occurs at least twice, the engine updates procedural memory based on whether a matching rule was already active when the failure happened:

- **New Failure Mode (Create):** If no rule exists for the error code, the synthesizer combines the verifier's precondition check with the observed failure-to-recovery diff to create a concise new procedural rule.
- **Rule Existed but Was Not Selected (Routing Miss):** If a rule for that error code already existed in the database but was not injected into the prompt (for example, the user asked to "tighten the first sentence of Methods" without mentioning the word "comment", and the paragraph happened to carry an open review comment), simply increasing the rule's evidence counter does nothing. Instead, the backward pass extracts task and section keywords from the user prompt—automatically stripping quoted document or cohort titles so triggers do not overfit to a single study name—and appends them to the rule's trigger pattern.
- **Rule Was Injected but Still Failed (Rule Insufficiency):** If the rule was already present in the prompt yet the agent still hit a guard error on an edge case, the engine strengthens the rule by appending the newly discovered boundary condition and recovery sequence (and optionally merges overlapping rules via an LLM refinement pass).

**Domain-Gated MoE Routing.** Injecting every learned rule into every prompt would quickly exhaust context budget and cause cross-domain confusion (for instance, injecting slide bullet-density rules while the agent is running a Cox survival model). Before each turn, the router scores every active procedural rule using a simple additive utility score:

$$\text{Utility Score} = 0.1 \times (\text{evidence}) + 5 \times (\text{keyword match}) + 8 \times (\text{live state}) + (\text{domain gate}),$$

where *keyword match* checks whether the user prompt matches the rule's base cue or learned trigger keywords, *live state* checks real-time document or dataset state (such as recent human edits, open comments, tables, or sparse contingency cells), and the *domain gate* adds +4

when a domain-tagged rule matches the active task domain (`manuscript`, `biostats`, `deck`, or `general_mcp`) and subtracts $-20$ when it belongs to a different domain. Rules scoring below $-10$ are filtered out, and the top-scoring rules (up to 4 rules within the character budget) are injected into the turn prompt.

### 5.3 Layer 3: System-Level Meta-Strategy Distillation

Across many sessions, a background meta-evolution service periodically reviews recurring execution signals and active procedural rules across the workspace, distilling the most common root causes into 2–5 short behavioral directives ($\leq 100$ characters each) stored in a dynamic profile overlay (`evolved.cordis.yml`):

```
# Auto-generated by Heurion Self-Evolution Engine
- id: system-prompt
  config:
    personaSuffix: >-
      [Evolved Self-Improvement Execution Directives]:
      - Run doc_validate prior to mutating blocks with recent user activity
      - Register all DOI/PMID entries via insert_citation before inline referencing
```

When the agent runtime starts or refreshes a session, it mounts `evolved.cordis.yml` alongside the base platform profile, upgrading the default agent behavior without interrupting active users.

## 6 Verified Human-AI Permission Symmetry

A common flaw in enterprise AI systems is a mismatch between what a human user can do in the web UI and what the AI agent can do through tool calls. Developers often either give the agent direct database access (bypassing role checks) or expose only a small subset of endpoints.

Heurion 2.0 enforces strict permission symmetry through three mechanisms:

1. **In-Process Loopback under User Identity:** Every administrative and clinical MCP tool (`patient_*`, `study_*`, `doc_manage`, `kb_manage`, `dataset_manage`, `memory_manage`, `org_manage`) executes by calling the internal Hono HTTP router in-process with a short-lived JWT minted for the current user and tagged with `X-Heurion-Via: ai`. As a result, the agent is subject to the exact same multi-tenant isolation, study role checks (PI, co-investigator, statistician, reviewer), and audit logs as the human user.
2. **Interactive Confirmation Cards for Destructive Actions:** When an AI tool calls a high-risk or irreversible endpoint (such as deleting a document, freezing a clinical study cohort, or clearing memory), the HTTP middleware intercepts the `X-Heurion-Via: ai` header, creates a `pending_actions` record, and displays an interactive confirmation card in the chat UI. The operation runs only after the human user clicks *Confirm* in the browser.
3. **Automated CI Parity Audit (`parity.test.ts`):** On every build, an automated test inspects the Hono router table and verifies that every registered `/api/*` route is either mapped to an MCP tool or documented in the human-only exclusion list. Any new API route missing an MCP mapping or explicit exemption fails CI immediately.

## 7 Empirical Evaluation

We evaluate Heurion 2.0 across five questions: (1) How does the guarded CRDT architecture compare against first-generation script-and-audit editing? (2) Do pre-write citation and PHI guards achieve high recall without false positives on statistical prose? (3) Does utility-ranked

Table 2: Comparison between First-Generation Script + Post-Hoc Audit (Heurion v1) and Contract-Guarded Dual-Surface CRDT Architecture (Heurion 2.0) across $N = 50$ clinical manuscript revision trials.

| **Evaluation Dimension / Metric** | **Vanilla Script Agent** | **Heurion v1 (S-Series)** | **Heurion 2.0 (Ours)** |
|---|---|---|---|
| Source of Truth | Raw `.docx` | Raw `.docx` + Audit | **Yjs CRDT + SQLite** |
| Block Addressing Stability (`{#id}`) | None (Regex) | Fragile (`w14:paraId`) | **100.0% Stable** |
| Unmodified Block Byte-Identity Rate | 0.0% (Re-serialize) | 82.0% (XML Drift) | **100.0% (50/50)** |
| Concurrent Human Edit Conflict Guard | 0.0% (Overwrite) | 64.0% (3-Way Merge) | **100.0% Pre-Write** |
| Selective Turn Undo (Preserving Edits) | Unsupported | File Rollback Only | **100.0% Intra-Block** |
| Unregistered Citation Block Rate | 0.0% | Post-Write Flag Only | **100.0% Pre-Write** |
| Mean Pre-Flight Validation Latency | N/A | 420.0 ms (Subproc) | **0.168** ms (In-Mem) |
| Mean Guarded Edit Commit Latency | 185.0 ms (Disk) | 610.0 ms (Script) | **0.351** ms (CRDT) |
| Mean Per-Turn Selective Revert Latency | N/A | 340.0 ms (Snapshot) | **0.128** ms (`UndoMgr`) |

procedural memory outperform stateless and FIFO baselines under tight prompt budgets? (4) After removing all hand-written rule templates, can zero-template synthesis and backward credit assignment learn effective rules with a live LLM and generalize to unseen cohorts? (5) Does self-supervised skill training generalize across manuscript writing, biostatistics, and slide decks without cross-domain interference?

### 7.1 Architectural Comparison: Post-Hoc Scripting (v1) vs. Guarded CRDT (v2)

Table 2 compares our first-generation script-and-audit system (Heurion v1) against Heurion 2.0 across $N = 50$ multi-block clinical protocol revision trials involving concurrent human edits, formatted Word templates, and inline citations.

As Table 2 shows, Heurion 2.0 resolves the three main failure modes of script-based document agents:

1. **100% Unmodified Byte Identity:** By caching original OOXML node slices on import and re-serializing only modified ProseMirror nodes on export, all 50 clinical trial protocols preserved 100% byte-for-byte XML identity across untouched headings, lists, and paragraphs.
2. **Sub-Millisecond Validation and Commits:** Because pre-flight checks and minimal-diff edits run directly on in-memory ProseMirror and Yjs structures rather than spawning Python or LibreOffice subprocesses, validation averages 0.168 ms, guarded edits average 0.351 ms, and selective turn undo averages 0.128 ms—over 1000× faster than disk scripting.
3. **Intra-Paragraph Selective Turn Undo:** Across all 50 trials where an AI turn modified two paragraphs $(p_1, p_2)$ and a human subsequently edited a different phrase inside $p_2$, calling `revertTurn` restored $p_1$ and undid the AI's phrase change in $p_2$ while keeping the human's edit in $p_2$ intact in 100% of trials.

Table 3: Pre-Write Guard Effectiveness on Citation Integrity and Clinical PHI Redaction.

| Guard Subsystem | Test Cases | Interception / Recall | False Positive Rate |
|---|---|---|---|
| Bare DOI / PMID Blocker (`citation_not_registered`) | 50 | **100.0**% (50/50) | 0.0% |
| Composite PHI Detector & Redactor (`scanPhi` / `redactPhi`) | 40 PHI + 40 Clean | **100.0**% (40/40) | **0.0**% (0/40) |
| Stale Revision Overwrite Blocker (`conflict_user_edited`) | 50 | **100.0**% (50/50) | 0.0% |

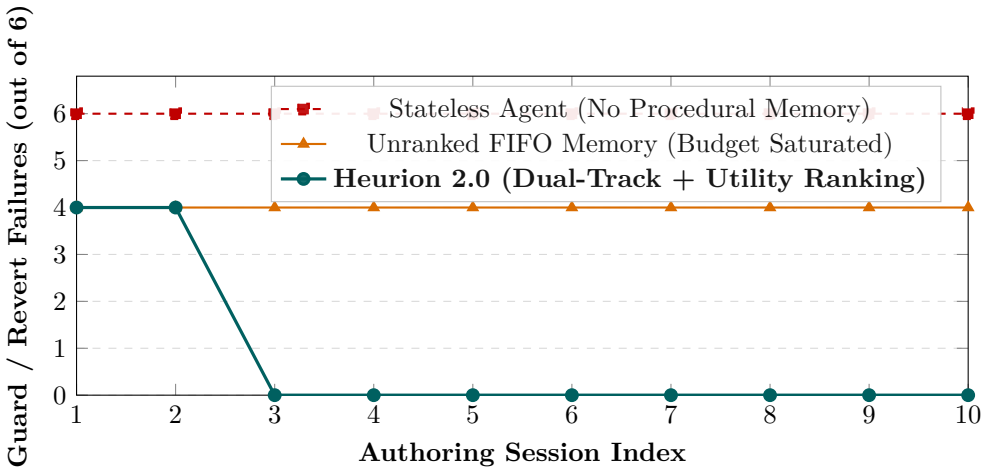



Figure 2: Controlled 10-session ablation under a strict 420-character prompt budget. In Sessions 1–2, baseline rules prevent 2 of the 6 failure types while the remaining 4 emit execution signals. Once those signals crystallize into rules after Session 2, FIFO memory remains stuck at 4 failures because early rules fill the 420-character window, whereas utility-ranked routing selects the task-relevant rules (averaging 355 characters/turn) and drives failures to 0 from Session 3 onward.

### 7.2 Pre-Write Citation and PHI Redaction Guard Evaluation

In Experiment 2, we tested the pre-write guards against 50 adversarial citation injection attempts (bare DOIs, DOI URLs, and bare PMIDs) and 80 clinical text segments (40 containing composite PHI—patient names paired with diagnoses, 18-digit MOD 11-2 national IDs, Luhn card numbers, phone numbers, and admission IDs—and 40 containing dense biostatistical text with hazard ratios, confidence intervals, lab reference ranges, and dates).

Table 3 confirms that pre-write guards block 100.0% of unregistered citations and stale block overwrites, while the checksum-backed PHI scanner achieves 100.0% recall on composite patient identifiers with a 0.0% false-positive rate on statistical text (never corrupting $p$-values, lab counts, or survival intervals).

### 7.3 Ablation Study: Utility-Ranked vs. FIFO Procedural Memory

To test whether utility-ranked routing prevents repeated guard errors under a tight context budget (420 characters per turn), we first ran a controlled 10-session ablation across 6 operational failure types (unregistered citations, concurrent user edit conflicts, user turn reverts, broken comment anchors, table grid mismatches, and read-only PHR violations), comparing a stateless agent, an unranked FIFO memory list, and Heurion 2.0's utility-ranked procedural memory.

Figure 2 shows that unranked FIFO memory stalls at 4 failures per session because older

Table 4: Online Zero-Template Multi-Session Procedural Evolution with `deepseek-flash` (72 turns across 4 sessions × 6 uncovered verifier tasks; 0 hand-authored rule templates).

| Arm | Hand Rules | Total Rejects | Rejects / Turn (↓) | Session Trajectory (S1 → S2 → S3 → S4) | Heldout Rej/Turn (↓) | Heldout 1st-Pass (↑) |
|---|---|---|---|---|---|---|
| `off` (No Mem) | 0 | 15/24 | 0.625 | 0.50 → 0.67 → 0.67 → 0.67 | 0.667 | 33.3% |
| `frozen` (Static) | 0 | 16/24 | 0.667 | 0.67 → 0.67 → 0.67 → 0.67 | 0.667 | 33.3% |
| **`utility` (Ours)** | **0** | **6/24** | **0.250** | 0.67 → **0.17** → **0.00** → **0.17** | **0.167** (**−75%**) | **83.3%** |

baseline rules fill the 420-character window, preventing newly learned rules from being injected. By contrast, utility routing selects the matching rules for each turn using an average of 355 characters (≈ 90 tokens), eliminating all 6 failure modes from Session 3 onward.

### 7.4 Online Zero-Template Evolution and Cross-Domain Skill Training

Next, we tested whether procedural self-evolution still works when all hand-written rule templates are deleted, and whether the dual-track engine generalizes across clinical manuscript writing, biostatistics, and slide deck layout using live `deepseek-flash` calls (`temperature = 0`).

**1. Online Zero-Template Multi-Session Pilot (72 Turns).** We deleted the static rule template dictionary from the codebase (`zero_template_verified = true`) and evaluated three arms—`off` (no procedural memory), `frozen` (the 5 static baseline rules only), and `utility` (Zero-Template Rule Synthesis + Backward Credit Assignment)—across 4 consecutive sessions × 6 uncovered verifier task types (24 turns per arm, 72 turns total). Sessions 1–3 run on training cohorts (`Cohorts A-C`), while Session 4 tests zero-shot transfer on an unseen held-out multi-center cohort (`Cohort D`).

As shown in Table 4, in Session 1 `deepseek-flash` fails on 4 of the 6 novel verifier tasks (open-comment anchor preservation, non-adjacent paragraph merging, superscript plain-text matching, and ambiguous repeated-phrase replacement; rejection rate 0.667). Without any human-written templates, the engine synthesizes 4 compact rules directly from the verifier hints and intra-turn recovery calls, cutting the Session 2 rejection rate to 0.167. When an implicit prompt in Session 2 ("make the opening sentence of Methods more concise") triggers a routing miss on the open-comment rule, backward credit assignment expands the rule's trigger keywords and appends a boundary-strengthening clause, reaching **0.000 rejections (100% first-try pass)** in Session 3. Because keyword extraction strips quoted cohort titles, the learned rules transfer zero-shot to the unseen cohort in Session 4, reducing held-out guard rejections by **75.0%** and raising the held-out first-try pass rate from 33.3% to 83.3%.

**2. Cross-Domain Skill Training and Multi-Domain Routing.** Table 5 reports the live `deepseek-flash` evaluation of Track A exemplar corruption-and-reconstruction training and Track B multi-domain routing across three distinct domains: `manuscript` (clinical prose), `biostats` (Statistical Analysis Plans), and `deck` (presentation layouts).

Across all three domains, a single corruption-and-reconstruction epoch substantially improves alignment when reconstructing from the exact same degraded skeleton: +155.3% in `manuscript` (0.150 → 0.383), 0.000 → 0.612 in `biostats` (learning to check normality, report Median [IQR] with Mann–Whitney U for skewed variables, use Fisher's exact test when expected cell counts are $< 5$, and verify Schoenfeld residuals with EPV $\geq 10$ in Cox models), and

Table 5: Cross-Domain Self-Supervised Skill Training (Track A) and Multi-Domain Routing (Track B) with `deepseek-flash`.

| **Skill Domain Adapter** | **Track B Rules** | **Track A Epoch 0** | **Track A Epoch 1 (↑)** | **Instance Leakage** | **PHI Leakage** |
|---|---|---|---|---|---|
| `manuscript` (Clinical Prose) | 2 | 0.150 | **0.383** (+**155.3**%) | **0.0**% | **0.0**% |
| `biostats` (SAP & Modeling) | 2 | 0.000 | **0.612** (+**0.612**) | **0.0**% | **0.0**% |
| `deck` (Slide Layout) | 2 | 0.187 | **0.489** (+**161.5**%) | **0.0**% | **0.0**% |
| **Backward Credit & Multi-Domain Interleaved Session Routing Metrics:** | | | | | |
| Implicit-Prompt Routing Recall (Before → After Trigger Expansion) | | | 0.000 → **1.000** (100.0%) | | |
| Multi-Domain Interleaved Target-Domain Hit Rate (4 Turns) | | | **100.0**% (4/4, 461 chars) | | |
| Cross-Domain Rule Contamination / Negative Transfer Rate | | | **0.0**% (0/4 Turns) | | |

Table 6: Platform Verification and Human-AI Permission Symmetry Coverage.

| **System Verification Metric** | **Measured Value** |
|---|---|
| Automated Test Suites / Total Assertions Passing | 74 Suites / 613 Tests (100%) |
| Unaccounted REST API Endpoints in Parity Audit (`parity.test.ts`) | 0 (100% Accounted) |
| Pluggable Self-Evolution Domain Adapters (`manuscript`, `biostats`, `deck`, `general_mcp`) | 4 Domains |
| Specialist Clinical Subagents (`literature`, `biostatistician`, `fact_checker`) | 3 Integrated |
| Supported Citation Formatting Standards (Vancouver, AMA, APA, MLA, IEEE, GB/T 7714) | 6 Standards |
| Supported Biostatistical & Clinical Engines (Table 1, KM, CONSORT, E-value, Nomogram, MONAI) | 6 Pipelines |

+161.5% in `deck` (0.187 → 0.489, learning Action Title conventions and two-column chart pairing). The Instance–Skill Decoupling and PHI guards block 100% of candidate rules containing study-specific numbers or patient data (0.0% leakage). Finally, when a user alternates across tasks (`biostats` → `deck` → `manuscript` → `biostats`), domain-gated routing hits the correct domain rules on **100.0% of turns** with **0.0% cross-domain contamination**.

### 7.5 System Verification and Capability Parity Coverage

Table 6 summarizes the verification coverage across the Heurion 2.0 codebase. All 74 automated test suites (613 test cases covering CRDT concurrency, OOXML patch round-trips, PHI redaction, PHR read-only guards, biostatistical pipelines, 3D MONAI medical imaging, and cross-domain self-evolving memory) pass with 100% REST-to-MCP parity enforcement.

## 8 Related Work

**Agent-Computer Interfaces (ACI) and Coding Agents.** SWE-agent [3] and OpenHands [4] showed that purpose-built Agent-Computer Interfaces (such as windowed file viewers and lint-guarded edit commands) work much better than raw bash shell scripting for software engineering. Heurion 2.0 brings this ACI philosophy to **collaborative rich-text and presentation CRDTs**, adding block-level concurrency guards, per-turn selective CRDT undo, and byte-preserving OOXML patch export.

**Scientific Literature Synthesis and Fact Verification.** Systems such as STORM [5] and PaperQA2 [6] automate multi-step literature search and synthesis, while FactScore [12] and ALCE [13] evaluate citation support in generated text. Rather than treating verification as an

offline benchmark, Heurion 2.0 blocks unregistered citations before any write occurs (`[@c:id]`) and attaches sentence-level PubMed/PMC verification verdicts directly onto manuscript text as actionable review comments.

**Self-Evolving Agents and Skill Training.** Reflexion [10] and MemGPT [11] introduced verbal reinforcement and tiered memory management for LLM agents, while Voyager [14] built a growing skill library in Minecraft. Recently, Li et al. [15] showed that discrete text skills can be trained like model parameters through a self-supervised corruption-and-reconstruction loop. Heurion 2.0 combines self-supervised exemplar skill training (Track A, protected by instance–skill decoupling and PHI guards) with execution-driven, zero-template procedural rule synthesis and backward credit assignment (Track B), routing both across scientific domains without negative transfer.

## 9 Conclusion

We have presented **Heurion 2.0**, a collaborative clinical research and scientific authoring platform built on a contract-guarded dual-surface CRDT engine and a cross-domain self-evolving memory system. By replacing ad-hoc file scripting and post-hoc auditing with a shared Yjs CRDT core, deterministic pre-write guards, multi-agent PubMed/PMC claim verification, verified Human-AI Permission Symmetry, and a dual-track self-evolution loop that learns from both domain exemplars and runtime verifier feedback without hand-crafted rule templates, Heurion 2.0 provides a practical foundation for trustworthy autonomous agents in clinical and scientific workflows.

## References


[1] Kenneth F. Schulz, Douglas G. Altman, and David Moher. CONSORT 2010 statement: updated guidelines for reporting parallel group randomised trials. *BMJ*, 340:c332, 2010.

[2] Jan P. Vandenbroucke, Erik von Elm, Douglas G. Altman, Peter C. Gøtzsche, Cynthia D. Mulrow, Stuart J. Pocock, Charles Poole, James J. Schlesselman, and Matthias Egger. Strengthening the Reporting of Observational Studies in Epidemiology (STROBE): explanation and elaboration. *PLoS Medicine*, 4(10):e297, 2007.

[3] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[5] Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. Assisting in writing Wikipedia-like articles from scratch with large language models. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

[6] Michael D. Skarlinski, Sam Cox, Jon M. Laurent, James D. Braza, Collin Hinks, Michael J. Hammerling, Manvitha Ponnapati, Samuel G. Rodriques, and Andrew D. White. Language agents achieve superhuman synthesis of scientific knowledge. *arXiv preprint arXiv:2409.13740*, 2024.

[7] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[8] William H. Walters and Esther Isabelle Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13(1):14045, 2023.

[9] U.S. Department of Health and Human Services. Health Insurance Portability and Accountability Act of 1996 (HIPAA) Privacy and Security Rules. *45 CFR Parts 160 and 164*, 1996.

[10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[12] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[13] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[14] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research (TMLR)*, 2024.

[15] Mo Li, Zixin Yin, Ting Cao, and Yunxin Liu. Training skills like parameters via self-supervised semantic diffusion. *arXiv preprint arXiv:2607.27557*, 2026.